WEEK 3 · LESSON 17 OF 24 · HANDOUT

Hyperparameter Tuning with Grid Search

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=GoI_qOgRqfA — open the lesson online to watch inside the course.

🎓 **Go deeper:** SciPy 2018 tutorial: Machine Learning with scikit-learn, Part 2 (scikit-learn core developers) — https://www.youtube.com/watch?v=gK43gtGh49o

🎓 **Go deeper:** freeCodeCamp: scikit-learn — full course — https://www.youtube.com/watch?v=pqNCD_5r0IU

Your goal in this lesson: Tell parameters from hyperparameters, tune a model with GridSearchCV and RandomizedSearchCV, and avoid tuning on the test set.

Parameters vs hyperparameters

- **Parameters** are learned from data during `fit`: the coefficients of a regression, the questions in a tree.
- **Hyperparameters** are settings **you** choose before training: k in k-NN, `max_depth` of a tree, C in an SVM, the learning rate in boosting.

Choosing hyperparameters by hand is slow and easy to get wrong. **Grid search** tries every combination with cross-validation and keeps the best.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

```
from sklearn.model_selection import train_test_split

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

GridSearchCV

```
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

pipe = make_pipeline(StandardScaler(), KNeighborsClassifier())
grid = {
    "kneighborsclassifier__n_neighbors": [5, 15, 25, 35, 51],
    "kneighborsclassifier__weights": ["uniform", "distance"],
}
search = GridSearchCV(pipe, grid, cv=5, scoring="roc_auc")
search.fit(X_train, y_train)
print(search.best_params_)
print("Best cross-validated AUC:", round(search.best_score_, 3))
```

- The grid has 5 × 2 = **10 combinations**; with 5 folds that is **50 trainings**.
- Inside a pipeline, a setting is named **step name + two underscores + setting** (`kneighborsclassifier__n_neighbors`).
- `scoring="roc_auc"` tells the search what "best" means. Choose the measure that matches your goal (accuracy, recall, f1, roc_auc, neg_mean_absolute_error…).

The best k-NN uses about **51 neighbours** weighted by distance, with a cross-validated AUC of about **0.86**.

Tune a random forest

```
from sklearn.ensemble import RandomForestClassifier

rf_grid = {
    "max_depth": [3, 6, None],
    "min_samples_leaf": [1, 5, 20],
    "max_features": ["sqrt", None],
}
rf_search = GridSearchCV(RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1),
                         rf_grid, cv=5, scoring="roc_auc", n_jobs=-1)
rf_search.fit(X_train, y_train)
print(rf_search.best_params_, round(rf_search.best_score_, 3))

results = pd.DataFrame(rf_search.cv_results_)
print(results[["params", "mean_test_score", "rank_test_score"]]
      .sort_values("rank_test_score").head(5))
```

18 combinations × 5 folds = 90 forests. The winner limits depth to 6 with at least 5 borrowers per leaf (AUC about **0.87**) — a regularised forest. Looking at the top five shows how **flat** the top is: several settings are almost equally good.

Random search: faster for big grids

Grids explode: 5 settings with 5 values each = 3,125 combinations. **RandomizedSearchCV** tries a fixed number of random combinations instead — usually almost as good, much faster.

```
from scipy.stats import randint
from sklearn.model_selection import RandomizedSearchCV

rand = RandomizedSearchCV(
    RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1),
    {"max_depth": randint(2, 15), "min_samples_leaf": randint(1, 40)},
    n_iter=15, cv=5, scoring="roc_auc", random_state=42, n_jobs=-1)
rand.fit(X_train, y_train)
print(rand.best_params_, round(rand.best_score_, 3))
```

Finally — and only once — the test set

```
from sklearn.metrics import roc_auc_score

best = rf_search.best_estimator_          # already re-trained on all training data
print("Test AUC:", round(roc_auc_score(y_test, best.predict_proba(X_test)[:, 1]), 3))
```

The test AUC (about **0.85**) is a little below the cross-validated 0.87. That is normal: picking the best of many settings is slightly optimistic. If you went back and tuned again after seeing the test score, the test set would no longer be honest.

Good habits

- Start with a **coarse** grid (0.01, 0.1, 1, 10, 100), then zoom in around the best value.
- Tune the settings that matter most (k for k-NN; depth and leaf size for trees; learning rate and number of trees for boosting; C for SVM and logistic regression).
- Put **all** preparation inside a pipeline so it is redone in every fold.
- Remember the gains are often small — better **data and features** usually beat better tuning.

Key terms

- **Hyperparameter** — a setting chosen before training.
- **Grid search** — try every combination with cross-validation.
- **Random search** — try a set number of random combinations.
- **best_params_ / best_score_ / best_estimator_** — the winning settings, their CV score and the re-trained model.
- **Scoring** — the measure the search optimises.

Remember this: Let GridSearchCV or RandomizedSearchCV choose hyperparameters by cross-validation, then report the winner's score on the untouched test set exactly once.

Name: Date:

Activity

Task 1

Tune `LogisticRegression` C over [0.01, 0.1, 1, 10, 100] inside a pipeline. Does tuning help much?

Task 2

Tune `GradientBoostingClassifier` over `learning_rate` [0.05, 0.1] and `n_estimators` [100, 300] with `max_depth=2`.

Task 3

Change `scoring` to `"recall"` for the forest search. Do the best settings change? Why?

Task 4

Explain to a teammate the difference between a parameter and a hyperparameter, with one example of each.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Which of these is a hyperparameter?
   - A. max_depth of a decision tree
   - B. The questions inside a trained tree
   - C. A regression coefficient
   - D. The predicted probability
2. A grid has 3 × 3 × 2 combinations and cv=5. How many models are trained during the search?
   - A. 45
   - B. 90
   - C. 5
   - D. 18
3. How do you name k-NN's n_neighbors inside make_pipeline for a grid?
   - A. pipeline_k
   - B. knn.n_neighbors
   - C. n_neighbors
   - D. kneighborsclassifier__n_neighbors
4. When is RandomizedSearchCV better than GridSearchCV?
   - A. Never
   - B. Only for regression
   - C. When there is no data
   - D. When the grid would be very large
5. Why is the final test score often a little below the best CV score?
   - A. CV leaks data
   - B. The test set is wrong
   - C. Random forests are unstable
   - D. Choosing the best of many settings is slightly optimistic
6. What should you NOT do after seeing the final test score?
   - A. Go back and tune again using the test set
   - B. Explain the results
   - C. Save the model
   - D. Write the report

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.